

UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL
Instituto de Informática
Departamento de Informática Aplicada

Aula 25: Revisão

Prof. Dr. Eder John Scheid



INF01202 - Algoritmos e Programação



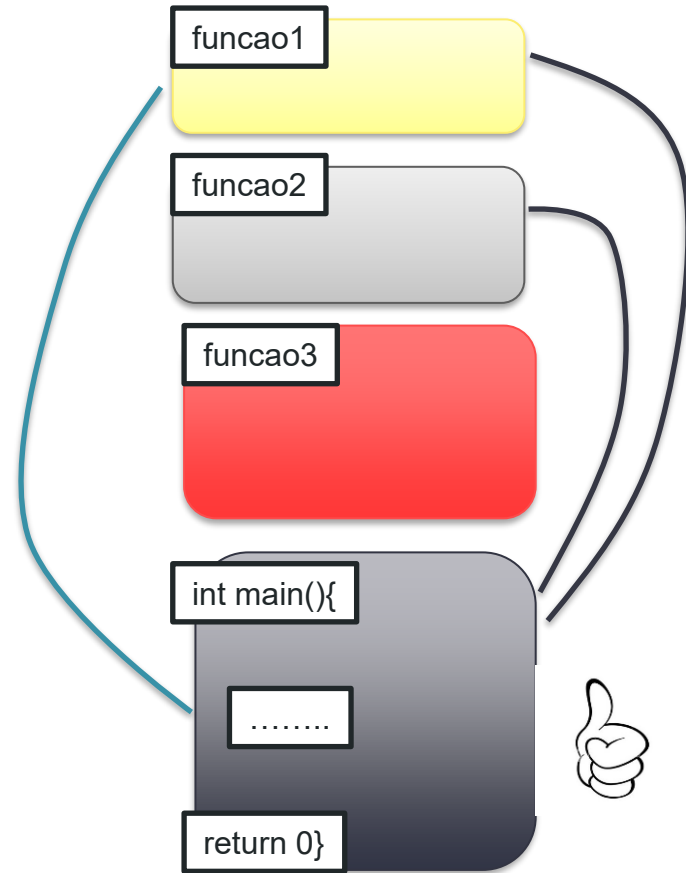
Funções ou subprogramas

- Sequência de instruções ou programas auto-contidos
- Uma função deve executar **UMA tarefa específica**, muito bem identificada
- Executados somente quando chamados por um programa em execução
- A execução de um programa em C corresponde a execução da função principal **main()**, que é obrigatória.
- Após sua execução, o fluxo do programa retorna ao ponto imediatamente após o da chamada da função (ou subprograma).

Idéia Principal



Programa sem estruturação



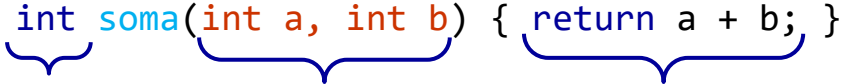
Programa estruturado

Função x Parâmetros

Quando um programa executa uma função:

1. A função pode receber dados deste programa, utilizados localmente para executar os comandos incluídos na função: estes dados são chamados de **parâmetros** ou **argumentos**.
2. Uma função pode também retornar um valor para o programa, o que chamamos de **retorno de função**.

Definição: `int soma(int a, int b) { return a + b; }`



Tipo da variável de retorno

Parâmetros

Instruções da função e retorno

Chamada: `int c; soma(10, 20); printf("%d", c);`



Argumentos

Tipos de Funções

Categoria	Protótipo	Chamada	Descrição
1. Sem retorno, sem parâmetros	<code>void mensagem(void);</code>	<code>mensagem();</code>	Executa algo, mas não recebe nem retorna valores
2. Sem retorno, com parâmetros	<code>void imprimir(int x);</code>	<code>imprimir(10);</code>	Executa algo com base em argumentos
3. Com retorno, sem parâmetros	<code>int pegarNumero(void);</code>	<code>int n = pegarNumero();</code>	Calcula algo e retorna, sem argumentos
4. Com retorno, com parâmetros	<code>float media(float a, float b);</code>	<code>m = media(5.0, 7.5);</code>	Calcula algo com base em argumentos e retorna resultado

Protótipos de Função em C

- Um protótipo de função é uma declaração antecipada da função, antes de sua utilização no código. Ele informa ao compilador:
 - O tipo de retorno
 - O nome da função
 - E os tipos dos parâmetros.
- Permite usar a função antes de defini-la
- Garante verificação de tipos durante a compilação
- Facilita modularização e organização do código
- Sintaxe: `tipo_retorno nome_funcao(tipo_param1, tipo_param2, ...);`
- Se definir a função antes do main, o protótipo é **opcional**

Exemplos

- Funções e seus protótipo:

```
void mostrarMensagem() {  
    printf("Olá,  
    mundo!\n");  
}
```



```
void mostrarMensagem(void);
```

```
int soma(int a, int b) {  
    return a + b;  
}
```



```
int soma(int a, int b);
```

```
float media(float x, float y, float z) {  
    return (x + y + z) / 3;  
}
```



```
float media(float x, float y, float z);
```

Passagem de parâmetros

- A **passagem de parâmetros** pode ser realizada de duas formas:
 - Por **valor**.
 - Por **referência**.

Passagem de parâmetros por valor

- Apenas um **valor** é passado para a função.
- Valores dos **parâmetros reais** utilizados na chamada são **copiados** para os **parâmetros formais** da função, que podem ser usados como **variáveis locais** da função.
- Alteração no valor dos parâmetros dentro do procedimento/função **não altera o valor** das variáveis passadas na chamada da função.

Passagem de parâmetros

- Desenvolver um procedimento que incrementa o valor recebido como parâmetro.

```
#include<stdio.h>
void incrementa(int x){
    x = x+1;
}

int main(){
    int n = 0;
    printf("Antes: %d\n",n);
    incrementa(n);
    printf("Depois: %d\n",n);
    return 0;
}
```

- O que será exibido em cada printf?
 - O valor de **n** não é alterado.
 - Passagem de parâmetros feita por valor.
 - Parâmetro **x** do procedimento recebe o valor de **n**.
 - Mudanças sobre a variável **x** são **locais** e **não afetam** o valor da variável **n** na função **main**.

Passagem de parâmetros por referência

- **Passagem por referência utiliza ponteiros.**
- Ao invés de **cópias** dos valores, são passadas **referências** (ponteiros) para estes valores
- Assim, os valores podem ser modificados dentro da função/procedimento.
 - Os **efeitos dessa modificação são propagados** para as variáveis da função que realizou a chamada da função.

Passagem por referência via **ponteiros**

- **Não** passa uma cópia do **valor**
- Passa uma **referência** ao valor na memória do computador
- Alterações feitas no parâmetro afetam o conteúdo **apontado** pela referência.
- Permite que funções/procedimentos façam **alterações** nos parâmetros, que são **propagadas** para a função/procedimento que realizou a chamada.
- Para isso, usamos **ponteiros**!

Passagem de Referência

- Desenvolver um procedimento que incrementa o valor recebido como parâmetro.

```
#include<stdio.h>
void incrementa(int *x){
    *x = *x+1;
}

int main(){
    int n = 0;
    printf("Antes: %d\n",n);
    incrementa(&n);
    printf("Depois: %d\n",n);
    return 0;
}
```

- O que será exibido em cada printf?
 - O valor de **n** é alterado.
 - **Passagem de parâmetros** feita por referência
 - Ponteiro **x** aponta para **n**.
 - Mudanças sobre o ponteiro ***x** **afetam** o valor da variável **n** na função **main**.

Passagem de parâmetros por referência

- Desenvolver um procedimento que incrementa o valor recebido como parâmetro.

```
#include<stdio.h>
void incrementa(int *x){
    *x = *x+1;
}

int main(){
    int n = 0;
    printf("Antes: %d\n",n);
    incrementa(&n);
    printf("Depois: %d\n",n);
    return 0;
}
```

Indica que a função recebe um **ponteiro** para **int**.

Indica que estamos passando um **ponteiro** para a variável **n** como **parâmetro real** para a função incrementa.

Aritmética de ponteiros

Adição e Subtração de ponteiros:

- Ponteiro armazena um endereço de memória.
- Podemos **somar** ou **subtrair** valores **inteiros** de **ponteiros**
- O valor do ponteiro irá **aumentar** ou **diminuir**, dependendo do **número de bytes** que o **tipo base** ocupa, isto é, o **endereço de memória apontado** será **deslocado** em função do **tamanho do tipo base**.

Regra geral:

$$[\text{<ponteiro> } +/- N] \rightarrow [\text{<endereço inicial> } +/- N * \text{<tamanho do tipo> }]$$

Aritmética de ponteiros

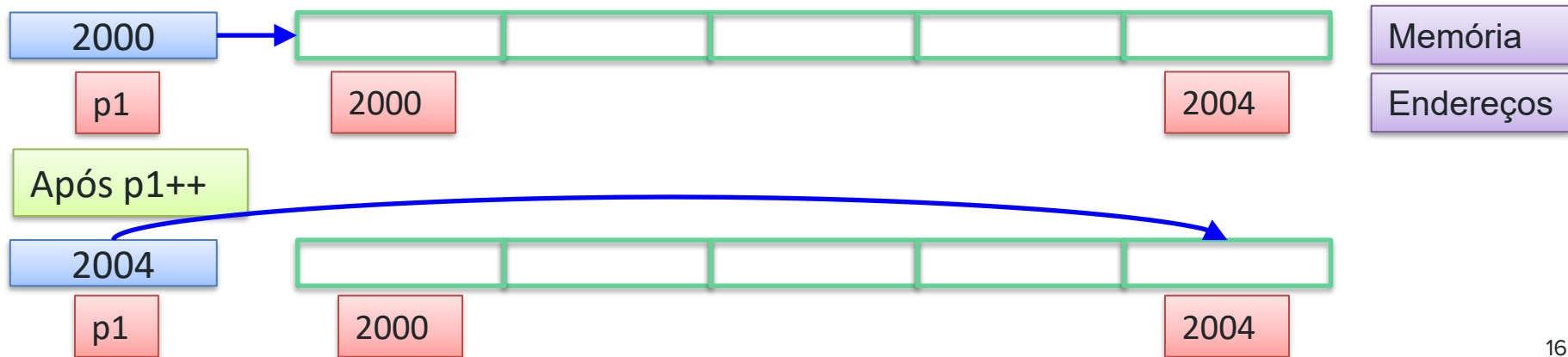
Supondo:

- Um ponteiro inteiro p1, apontando para o endereço de memória 2000.
- A variável inteira ocupando 4 bytes (pode variar dependendo da arquitetura).

`p1++;` // valor de p1 fica 2004 *

`p1--;` // valor de p1 fica 1996

`p1 = p1 + 5;` // p1 passa a apontar para $2000 + (5 \times 4)$



Relações entre ponteiros e vetores

- O nome de um vetor corresponde a um **ponteiro para o seu primeiro elemento**.

`[int n[50];]`  `n` é um ponteiro equivalente a `&n[0]`.

- Diversas operações válidas para ponteiros podem ser executadas com vetores.
- Assim, as notações abaixo são equivalentes:

Assumindo: `int v[100];`

Com vetores
conteúdo: `v[i]`
endereço: `&v[i]`



Com ponteiros
`*(v+i)`
`v+i`

declaração de vetor `v` de inteiros com 100 elementos indica que `v` é um ponteiro que aponta para o 1º elemento do vetor (`v[0]`), alocando a área necessária

declaração do ponteiro `v` aponta para a posição inicial, mas não aloca área para isso.

Passagem de arranjos como parâmetros

- Vetores **não podem** ser passados por **valor**.
 - Isso acarretaria cópia.
 - Seria dispendioso copiar vetores muito grandes (com milhões de posições).
 - Tanto em termos de tempo de processamento quanto de memória.
- Vetores são **sempre** passados por **referência**.
 - Modificações realizadas no vetor, dentro da função que o recebeu como parâmetro, são refletidas no vetor passado como parâmetro pela função que realizou a chamada.
- Apenas o **endereço do primeiro elemento do vetor** é passado
 - (**v**, ou seja, **&v[0]**)
 - A função que recebe o vetor não tem como saber qual é o tamanho dele sem informação adicional.
 - Por isso, para construir **funções genéricas**, capazes de lidar com qualquer vetor (de qualquer tamanho) passado como parâmetro, também precisamos passar o **tamanho** do vetor como parâmetro para a função.

Passagem de arranjos como parâmetros

- Declaração de um parâmetro **vetor** em uma **função/procedimento**.

`<tipo> <nome do vetor>[]`
`<tipo> *<nome do vetor>`

Equivalentes

- Onde:

- **<tipo>**: tipo dos elementos do vetor
- **<nome do vetor>**: nome atribuído ao vetor
- **[]**: indica que a variável é do tipo vetor
- *****: indica que é um ponteiro para o tipo primitivo de dado do vetor

Exemplo usando a notação de ponteiro

```
#include<stdio.h>
#define QTD1 5
#define QTD2 10
void inicializa(int *v, int n, int tamanho){
    int i;
    for(i=0; i<tamanho; i++){
        v[i] = n;
    }
}
void imprime(int *v, int tamanho){
    int i;
    for(i=0; i<tamanho; i++){
        printf("%d\t", v[i]);
    }
    printf("\n");
}
int main(){
    int v1[QTD1], v2[QTD2], i;
    inicializa(v1, 1, QTD1);
    inicializa(v2, 2, QTD2);
    imprime(v1, QTD1);
    imprime(v2, QTD2);
    return 0;
}
```

Equivalente a

`int v[]`

Arranjos multidimensionais como parâmetros

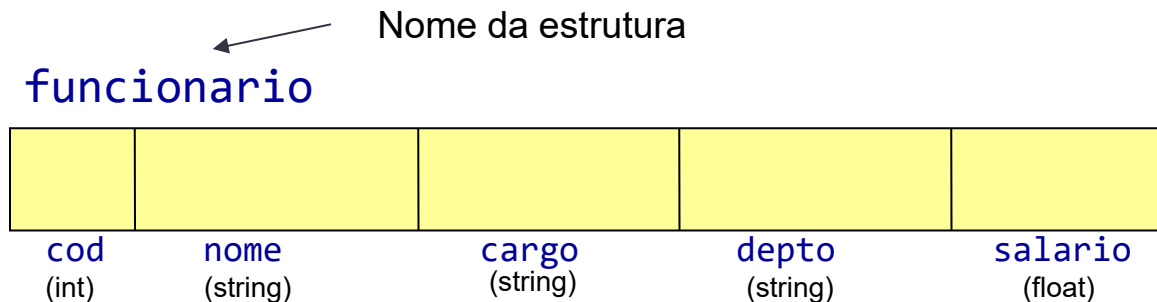
- Passagem de arranjos com mais de uma dimensão em C é um pouco mais complexa.
- A alternativa mais simples é **fixar o tamanho** de todas as dimensões (com exceção da primeira) na declaração da função.
- Há outras possibilidades.
 - Fora do escopo da disciplina.

Exemplo:

```
void funcao1 (float array1[][10]);  
void funcao2 (int array2[][20][30]);  
/*a primeira dimensão não precisa ser especificada,  
mas as demais DEVEM!*/
```

Estruturas

- Tipo de dado, que permite agrupar informações com características diferentes num único local
- Elementos com tipos de dados diferentes.
- Cada unidade de informação é um campo da estrutura
- Exemplo:



5 campos: cod, nome, cargo, depto, salario
Cada campo tem um tipo (int, float, char[])

Diferença entre arranjo (array) e estrutura

Arranjo (homogêneo)

- Todos os elementos do mesmo tipo.
- Cada elemento é identificado através do mesmo nome acrescido de um índice, para identificar este elemento: variável indexada.

Estrutura (heterogênea)

- Elementos podem ser de tipos diferentes.
- Cada elemento é identificado através de um nome único - campo.

Declaração de tipo estrutura

Sintaxe

```
struct identificador{  
    tipo1  campo1, campo2;  
    tipo2  campo3;  
    ...  
    tipon  campon;  
};
```

O que está sendo declarado em cada exemplo é um **tipo estrutura!**

Ainda não existe qualquer variável declarada que utilize este tipo (no próximo slide)

Exemplos

```
struct str_funcionario{  
    int      cod;  
    char     nome[30];  
    char     cargo[10];  
    int      depto;  
    float    salario;  
};
```

```
struct str_numeros{  
    int c1;  
    float c2;  
    float c3;  
};
```


Exemplo

```
#include <stdio.h>
```

```
struct str_funcionario{ // declaração de tipo: é global
    int    cod;
    char   nome[30];
    char   cargo[10];
    int    depto;
    float  salario;
};
```

A ordem segue a ordem da declaração da estrutura

```
int main(){
    // declaração da variável, já inicializada
    struct str_funcionario funcionario = { 1432, "Jose Costa", "Gerente", 2, 4656.00 };
    .....
}
```



Acesso aos campos de uma estrutura

Campos são acessados individualmente com a notação:

nome da variável.nome_do_campo

```
struct str_numeros{
    int c1;
    float c2;
    float c3;
};

int main(){
    struct str_numeros tres_num;

    tres_num.c1 = 10;
    tres_num.c2 = 35.7;
    tres_num.c3 = 7.5 * tres_num.c1;
    ...
    scanf("%d", &tres_num.c1);
    printf("%d", tres_num.c1);
    ...
}
```

Ponto para separar o nome da variável do nome do campo

Comando typedef

A linguagem C provê um mecanismo para criarmos novos nomes para tipos de dados com o comando **typedef**

Facilita a legibilidade de programas, por exemplo, no exercício anterior usávamos a seguinte definição:

```
typedef struct struct_aluno{  
    char nome[15 + 1];  
    float nota[NRNOTAS];  
    float media;  
    char conceito;  
}ALUNO;
```

Neste exemplo a declaração:

```
struct struct_aluno aluno[NRALUNOS];
```

Seria substituída por:

```
ALUNO aluno[NRALUNOS];
```



As maiúsculas não são necessárias, mas ajudam a lembrar que este é um tipo “criado” pelo programador e não nativo do C

Typedef – Exemplos

```
typedef struct{  
    int dia;  
    char mês [3+1] ;  
    int ano;  
}DATA;
```

```
typedef struct s_pessoa{  
    char nome [60] ;  
    float salario;  
    DATA nasc;  
}PESSOA;
```

```
/* Vetor de Estruturas */  
struct s_pessoa mulher[3];  
ou  
PESSOA mulher [3];
```

```
/* Inicializando */  
PESSOA mulher[3] = {  
    {"Ana", 5400.00, {2, "Jan", 1980} },  
    {"Joana", 400.00, {28, "Dez", 1989}  
},  
    {"Laura", 2100.00, {2, "Jul", 1979} },  
};
```

Passagem de estruturas por referência

Ao passarmos uma estrutura por referência teremos o seguinte

```
void le_pessoa(PESSOA *ptr){ //incompleto, apenas exemplo
    printf("Entre com o salario: ");
    scanf("%f", &(*ptr).salario);
    . . .
}
```



Vejam a necessidade dos parênteses, pois o operador Ponto tem maior precedência do que o Ponteiro

Sem os parênteses, o compilador vai interpretar como `&*ptr.salario`, mas `ptr` não é do tipo estrutura, é tipo ponteiro para estrutura. ERRO!!

Passagem de estruturas por referência

A necessidade de colocar os parêntesis torna a notação de difícil leitura, apesar de funcionar. Para tornar a leitura mais limpa, C tem o Operador SETA (->). Veja abaixo como fica:

```
void le_pessoa(PESSOA *p){ //incompleto, apenas exemplo
    printf("Entre com o salario: ");
    scanf("%f", &(*p).salario);
    . . .
}

void le_pessoa(PESSOA *p){ //incompleto, apenas exemplo
    printf("Entre com o salario: ");
    scanf("%f", &p->salario);
    . . .
}
```



EQUIVALENTES!!

Operador seta: ->

- Utilizado em funções com parâmetro do tipo estrutura, onde seja necessário o uso de ponteiros (parâmetro passado por referência).
- Indica o acesso a um campo de uma estrutura por meio de um ponteiro para esta estrutura.
- Escrever **aluno-> media** equivale a escrever **(*aluno).media**
- O parênteses é necessário porque o ● (ponto) tem prioridade maior que o *.
- Atenção: a notação com o operador seta é a de uso mais frequente.

Exemplo - Função de leitura usando o operador seta

```
void le_aluno (struct struct_aluno *aluno){
    int i;

    printf("\nNome: ");
    fgets(aluno->nome, 15, stdin); // recebe nome do aluno
    aluno->nome[strlen(aluno->nome)-1] = '\0';
    aluno->media = 0; // zera a media
    for (i = 0; i < NRNOTAS;i++){
        printf("\nNota %d: ", i + 1);
        aluno->nota[i] = ler_nota(); // leitura com scanf
        aluno->media = aluno->media + aluno->nota[i];
    }
    aluno->media = aluno->media / NRNOTAS; // calcula a media
    aluno->conceito = conceito(aluno->media); // atribui conceito
}
```

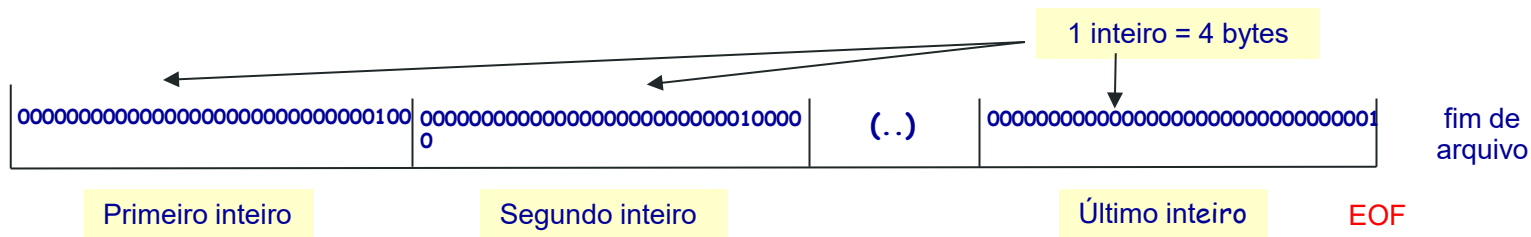

Arquivos

- Conjuntos de dados identificados por um nome e que podem ser acessados através de programas
- Usualmente armazenados em dispositivos de memória auxiliar (discos)
- Permanentes



Tipos de arquivos do C

- **Binário:** composto por uma sequência de bytes lidos, sem tradução, diretamente do dispositivo externo. Não ocorre nenhuma tradução e existe uma correspondência um para um entre os dados do dispositivo e os que estão no fluxo



- **Texto:** composto de uma sequência de bytes correspondendo a uma sequência de caracteres ASCII, incluindo letras, dígitos e caracteres especiais utilizados na escrita e os separadores branco, tab e new line

'A' 'u' 'l' 'a' '_' '2' '5' '\n' EOF

Esquema de utilização de arquivos em C

A utilização de arquivo em C envolve os seguintes passos:

1. Declaração do arquivo através de um ponteiro na forma

```
FILE *point_arq;
```

2. Abertura do arquivo, onde o ponteiro declarado é associado fisicamente ao arquivo externo através de

```
point_arq = fopen (“nome_do_arq”, “modo abertura”);
```

3. Operações com arquivo, através de funções de manipulação de arquivos (aplicadas ao ponteiro)

4. Fechamento do arquivo, através de

```
fclose (point_arq);
```

Modos de abertura para arquivos binários

Modo de abertura	Arquivo Existe	Arquivo não existe	Permissão de leitura	Permissão de escrita
rb	OK	Erro	✓	
wb	Cria novo, antigo eliminado	Cria		✓
r+b ou rb+	OK	Erro	✓	✓
w+b ou wb+	Cria novo, antigo eliminado	Cria	✓	✓
ab	Novos dados acrescentados ao final	Cria		✓
a+b ou ab+	Novos dados acrescentados ao final	Cria	✓	✓

O **b** significa binário

O **+** significa leitura e escrita

O **r** significa *read*

O **w** significa *write*

O **a** significa *append*

Funções de manipulação de arquivo

```
...  
FILE *arq;  
arq = fopen("exemplo.bin","rb"); // se ocorrer erro, arq recebe NULL  
if (!arq)  
    printf("Erro na abertura do arquivo.");  
    /* na sequência encerra a execução do programa */  
else{  
    /* efetua operações com arquivo */  
    ...  
    fclose(arq); /* fecha o arquivo */  
}  
...
```

Funções de manipulação de arquivos binários

```
fread (&varbuffer, numbytes, quant, point_arq);
```

Lê dados do arquivo para a memória principal

Se valor retornado $\neq$ quant, em geral é indicação de erro!

-> SEMPRE testar o valor retornado pela função!

- lê o dado a partir do elemento corrente e o armazena no endereço `&varbuffer`
- `numbytes` é o tamanho da unidade a ser lida
- `quant` indica quantas unidades de `numbytes` serão lidas: logo, o número total de bytes que poderão ser lidos é igual a `numbytes*quant`
- `varbuffer` pode ser do mesmo tamanho que `numbytes` ou um ponteiro para uma região da memória;
- a quantidade - `quant` - de `numbytes` efetivamente lida é retornada pela função, podendo ser menor se o final do arquivo for alcançado previamente ou se ocorrer erro.

Funções de manipulação de arquivos binários

```
fwrite (&varbuffer, numbytes, quant, point_arq);
```

Grava dados no arquivo

Se valor retornado for $\neq$ quant, em geral é indicação de erro!

- **SEMPRE** testar o valor retornado pela função!

- o funcionamento é similar à função de leitura, só que para escrita
- a quantidade - quant - de numbytes efetivamente escrita (podem ocorrer erros ou buffer estar incompleto) é retornada pela função.

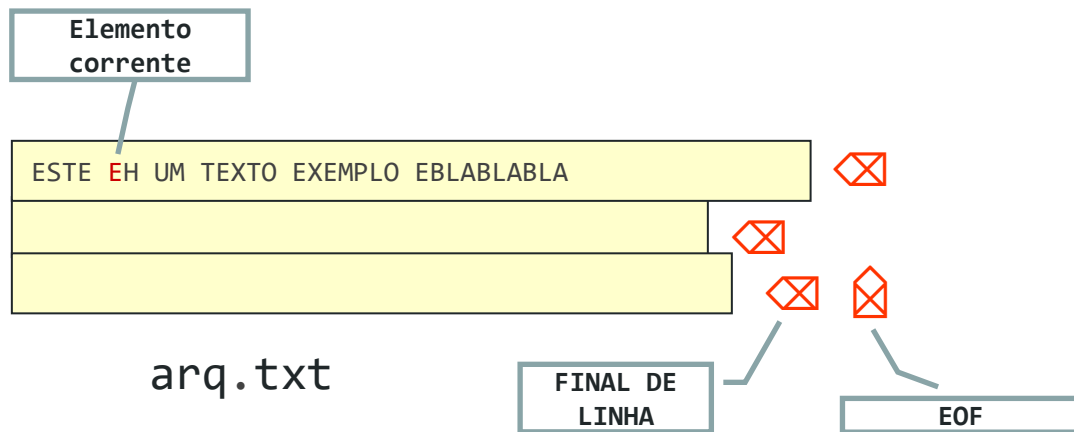
Arquivo Texto: organização interna

Um arquivo texto compõe-se de uma sequência de caracteres, formando linhas de tamanho variável.

O elemento corrente de um arquivo texto é o caractere em que o sistema ficou posicionado após a última operação sobre o arquivo.

As mudanças de linha são geradas pela presença do caractere '\n'

O final de arquivo é sinalizado por um caractere indicador de fim de arquivo que, internamente aos programas, é identificado como EOF.



Abertura de arquivo (binário ou texto)

```
point_arq = fopen (nome do arquivo, modo de abertura);
```



Semelhante ao arquivo binário, mas sem incluir a letra 'b'

- Abre/cria/recria um arquivo com o nome (completo, do arquivo físico) e modo de abertura desejado passados como parâmetro do tipo **string**
- A função retorna um apontador para a estrutura FILE. Se não for possível abrir o arquivo, ocorre erro e o valor retornado em **point_arq** é NULL

OBS: após abertura, o programa deve sempre verificar se não houve erro!

Funções de gravação de arquivo texto, por grupo de caracteres

fprintf (**point_arq**, **formatos**, **lista de variáveis**);

Escreve caracteres em um arquivo, baseado em formatos, de maneira similar ao **printf()**. Para quebra de linha é necessária a inclusão de '**\n**' no formato do comando

Retorna o número de caracteres escritos (ou número negativo em caso de erro)

Funções de gravação de arquivo texto, por grupo de caracteres

fputs (buffer,point_arq)

Escreve o conteúdo da variável **buffer** no arquivo apontado por **point_arq**. Grava até atingir o **'\0'** mas **não grava ele** no arquivo, apenas **'\n'**, se existir

Retorna valor positivo (inteiro) em caso de sucesso, ou EOF no caso de erro.

Funções de leitura de arquivo texto por linha

fscanf (**point_arq**, **formatos**, **&variaveis**)

Lê uma string/linha de um arquivo, baseado em formatos, semelhante ao **scanf**

Retorna o número de elementos lidos com sucesso (valor inteiro), ou -1 se nenhum elemento for lido.

Funções de leitura de arquivo texto por linha

fgets(&buffer_str, num, point_arq)

Lê uma string/linha de um arquivo, até encontrar **'\n'** ou EOF ou completar **num-1** caracteres.

Obs: a função mantém na leitura o **'\n'** de fim da linha, se existir e couber, incluindo na string, e utiliza o último caractere reservado para o terminador de string.

Retorna um ponteiro para char (valendo NULL se EOF for lido antes de qualquer caracter)

Comparando fscanf e fgets

```
#include <stdio.h>
#define NUMCHAR 256

int main( ){
    FILE *arq;
    char buffer[NUMCHAR];

    if(!(arq = fopen("teste.txt","r"))) // abre para leitura
        printf("Erro na abertura\n");
    else{
        printf("Imprimindo conteudo do arquivo com fscanf\n");

        while(!feof(arq)){
            if (fscanf(arq,"%s",buffer) != -1) // garante leitura correta
                printf("%s-",buffer);
        }
        rewind(arq);

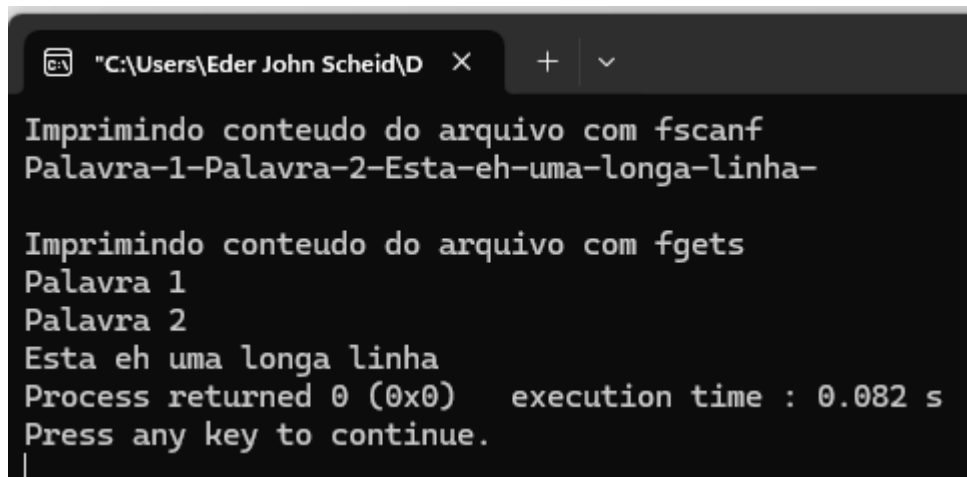
        printf("\n\nImprimindo conteudo do arquivo com fgets\n");

        while(!feof(arq)){
            if (fgets(buffer,256, arq) != NULL ) // garante leitura correta
                printf("%s",buffer);
        }
        fclose(arq);
    }
    return 0;
}
```

Arquivo Teste.txt

Palavra 1
Palavra 2
Esta eh uma longa linha

Exemplo de execução



```
"C:\Users\Eder John Scheid\D" X + v
Imprimindo conteudo do arquivo com fscanf
Palavra-1-Palavra-2-Esta-eh-uma-longa-linha-

Imprimindo conteudo do arquivo com fgetc
Palavra 1
Palavra 2
Esta eh uma longa linha
Process returned 0 (0x0)    execution time : 0.082 s
Press any key to continue.
|
```

Funções de gravação de arquivo texto, caractere a caractere

putc (**caractere**, **point_arq**)

Escreve um *caractere* no arquivo. Para indicar quebra de linha, precisa gravar o caractere de quebra de linha ('\n'). Para inserir final de arquivo, gravar o caractere por **EOF** ou **fechar o arquivo**.

Retorna o código ASCII (inteiro) do caracter escrito em caso de sucesso, ou -1 em caso de erro.

- **Boa prática de escrever stream para arquivo com fflush(**point_arq**) após um putc**

Funções de leitura de arquivo texto, por caractere

```
ch =getc (point_arq)
```

Retorna um caractere, lido do arquivo. Se o final do arquivo for encontrado ou ocorrer erro, a função retorna **EOF**.

Exemplo: getc e putc

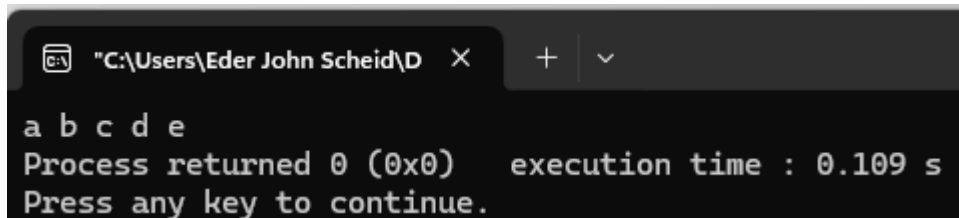
```
#include <stdio.h>

int main( ){
    FILE *arq;
    char letra,ch;

    if(!(arq = fopen("Letras.txt","w+"))) // abre para escrita e leitura
        printf("Erro na abertura\n");
    else{          // grava caracteres 'a' à 'f' e insere fim de arquivo
        for(letra='a'; letra < 'f'; letra++)
            putc(letra,arq);
        putc EOF,arq); // insere marca de fim de arquivo
        rewind(arq);

        // lê arquivo texto até marca de fim do arquivo ou erro de leitura
        ch = getc(arq);

        while (ch != EOF){
            printf("%c ", ch); // imprime caractere na tela
            ch = getc(arq);
        }
        fclose(arq);
    }
    return 0;}
```



```
"C:\Users\Eder John Scheid\D  X  +  v
a b c d e
Process returned 0 (0x0)   execution time : 0.109 s
Press any key to continue.
```

Acesso Randômico

- Acesso a pontos de um arquivo, **sem percorrer sequencialmente o conteúdo que antecede o conteúdo desejado.**
- **Motivações**
 - a posição **desejada** de leitura ou escrita **pode não ser** coincidente com a **posição corrente** do arquivo válida em um certo momento.
 - A **posição corrente** de leitura ou escrita **pode ser alterada** para qualquer posição do arquivo.

Mais funções de manipulação de arquivo

- `rewind`

- `fseek`

- `ftell`

Funções de manipulação de arquivos

`rewind(point_arq)`

Coloca a posição corrente do arquivo para o início do arquivo

-> Usar apenas quando, após a abertura do arquivo, tiverem sido executadas funções de leitura ou gravação, que tenham deslocado a posição corrente para depois do início do arquivo!

```
FILE *arq;  
arq = fopen(...);  
...  
// funções de leitura e/ou gravação de arquivo  
rewind(arq);  
// posição corrente = início  
...
```

Funções de manipulação de arquivos

Pode ser negativo

`fseek(point_arq, numbytesdesloc, origem)`

Move a posição corrente de leitura/escrita para a posição calculada **origem + numbytesdesloc**. Os valores possíveis, definidos, para **origem** do deslocamento são:

SEEK_SET ou **0** = início do arquivo

SEEK_CUR ou **1** = ponto corrente no arquivo

SEEK_END ou **2** = fim do arquivo

-> Usar apenas quando quiser **alterar o acesso sequencial** ao arquivo, que ocorre quando do uso consecutivo de funções de leitura ou gravação de arquivo!

ATENÇÃO: fseek SÓ POSICIONA. NÃO LÊ, NEM GRAVA !

```
...  
FILE *arq;  
struct estrutura buffer;  
arq = fopen(...);  
/*coloca a posição para o início na  
2a estrutura do arquivo*/  
fseek(arq, sizeof(struct estrutura), SEEK_SET);  
fread(&buffer, ..., ..., arq); // lê 2o registro  
...
```

Funções de manipulação de arquivos

`ftell(point_arq)`

Indica a posição corrente no arquivo.

-> Posição dada **em bytes**.

```
...
FILE *arq;
arq = fopen(...);
...

/* posição corrente = início */
printf("\nposicao atual em bytes antes leitura: %d\n", ftell(arq));

if (fread(&buffer, sizeof(TIPO_ATLETA), 1, arq) == 1)
    //lê, confirma se o que foi lido conteúdo e não EOF, então imprime:
    printf("\nposicao atual em bytes apos leitura: %d\n", ftell(arq));
...
...
```

Outras Funções de manipulação de arquivo texto

`ferror(point_arq)`

Indica se a última operação com um arquivo produziu um erro

Retorna verdadeiro (não-zero) se ocorrer algum erro, caso contrário, retorna falso (zero)

Cada nova operação com o arquivo modifica a condição de erro

`rename(nomeantigo,novonome)`

Renomeia um arquivo ou o **move** para um diretório destino

Obs.: o arquivo deve estar fechado!

`remove(nomedoarquivo)`

Apaga o arquivo especificado do disco

Obs.: o arquivo deve estar fechado!

UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL
Instituto de Informática
Departamento de Informática Aplicada

Obrigado pela atenção!
Dúvidas?

Prof. Dr. Eder John Scheid



INF01202 - Algoritmos e Programação
(Material adaptado dos slides do professor Marcelo Walter, Renan Maffei, e
Leonardo Gobatto)

